Task 01: The device sigil

The first piece of the engine: work out what a device can run before we download anything.

Owner	Sunaina
Size	Small, about 2 to 3 days
Where	`circle/src/sigil/`
Build it in	Kiro, Spec mode, on branch `feature/device-sigil`

What and why

Before the Circle downloads a model it must know what the device can handle. A gaming laptop can run a big model; an old phone can't. Guess wrong and the page crashes, or we download 2 GB for nothing.

The **sigil** looks at the browser once and answers two questions:

- **What does this device have?** Graphics card, memory, free storage.
- **Which model size should we use?** `tiny`, `small`, `full`, or none if nothing fits.

Everything else in the engine depends on this answer. It is small and self-contained: no AI models, no network, just browser APIs and logic.

Files to create

```
circle/
  package.json        name "circle", type "module", scripts: test, typecheck
  tsconfig.json       strict mode
  vitest.config.ts
  src/
    index.ts          exports probeDevice, chooseTier and the types
    sigil/
      types.ts        DeviceFacts, Tier, Sigil, TierReason
      probe.ts        probeDevice(): reads the browser
      tier.ts         chooseTier(): pure decision logic
      probe.test.ts
      tier.test.ts
```

Also add `"circle"` to `workspaces` in the root `package.json`.

The contract (types)

Other parts of the engine will rely on these exact shapes.

```ts
export type Tier = "none" | "tiny" | "small" | "full";

/** Raw facts from the browser. undefined = the browser didn't say. */
export type DeviceFacts = {
  webgpu: boolean;         // a real (not software) WebGPU adapter was found
  gpuName?: string;        // e.g. "intel gen-12lp"
  maxBufferMB?: number;    // largest GPU buffer the adapter allows
  memoryGB?: number;       // navigator.deviceMemory (Chrome only, capped at 8)
  cores?: number;          // navigator.hardwareConcurrency
  freeStorageMB?: number;  // storage quota minus what is already used
};

export type TierReason =
  | "NO_WEBGPU" | "LOW_MEMORY" | "LOW_STORAGE"
  | "SMALL_GPU_BUFFER" | "UNKNOWN_MEMORY";

export type Sigil = {
  facts: DeviceFacts;
  tier: Tier;
  reasons: TierReason[];   // why a higher tier wasn't picked (empty if "full")
};
```

The functions

```ts
/** Reads the browser. Never throws: a failing or missing API becomes undefined. */
probeDevice(env?: BrowserEnv): Promise<Sigil>

/** Pure decision. Same facts in, same answer out. No browser access. */
chooseTier(facts: DeviceFacts): { tier: Tier; reasons: TierReason[] }
```

The key design idea: `BrowserEnv` is a small object holding the browser parts `probeDevice` needs. The real browser is used by default; tests pass in a fake one to simulate any device, from a gaming laptop to a broken browser.

Tier rules

Check from the top. The first row that fits wins.

Tier	Needs all of these
full	WebGPU, memory 8 GB or more (known), free storage 4,000 MB or more, GPU buffer 1,024 MB or more
small	WebGPU, memory 4 GB or more **or unknown**, free storage 1,000 MB or more
tiny	Free storage 200 MB or more (runs on the CPU, no WebGPU needed)
none	Anything else

- **Unknown memory** is allowed for `small` (Safari and Firefox don't report it) but adds `UNKNOWN_MEMORY`.
- **Unknown storage or GPU buffer** counts as not enough for the tier that needs it.
- For each requirement a higher tier failed, add its reason. **Example:** WebGPU, 8 GB memory, but only 1.5 GB free storage gives `small` with `["LOW_STORAGE"]`.

Browser APIs

Check each one in the current official docs (MDN, the WebGPU spec) before using it, and write what you confirm into `.kiro/steering/local-ai.md`. Our project rule: no guessed APIs.

- `navigator.gpu.requestAdapter()`, then the adapter's `limits.maxBufferSize` and `info`, and how the spec marks a software fallback adapter (which counts as no WebGPU).
- `navigator.deviceMemory` and `navigator.hardwareConcurrency`
- `navigator.storage.estimate()`: free storage is `quota - usage`

A simpler early version lives in `site/src/probe.ts`. Read it for reference, then write the new one properly to this spec.

Tests

Vitest for unit tests, fast-check for property-based tests. Pin exact versions.

Unit test (a specific device)	Expected
Strong gaming laptop	full, no reasons
WebGPU, 8 GB, 1.5 GB free storage	small, [LOW_STORAGE]
Safari-like: WebGPU, memory unknown, lots of storage	small, [UNKNOWN_MEMORY]
Old phone: no WebGPU, 500 MB free	tiny, [NO_WEBGPU]
Storage almost full: 50 MB free	none
Software fallback adapter	treated as webgpu: false
Fake env where every API throws	returns a valid sigil, never throws

Property-based tests (fast-check generates thousands of random devices):

- ✓ **More never hurts:** if device B has at least as much of everything as A, B's tier is never lower. (WebGPU beats no WebGPU. Only compare a value that is known on both or unknown on both.)
- ✓ **No WebGPU, no GPU tiers:** without WebGPU the tier is always tiny or none.
- ✓ **Fits the device:** the chosen tier's storage need is never more than the free storage.
- ✓ **Deterministic:** the same facts always give the same answer.
- ✓ **Honest reasons:** full always has no reasons; every other tier has at least one.

How to do it in Kiro

1. Pull the latest `main`, create branch `feature/device-sigil`.
2. Open Kiro in **Spec mode** and paste:

> *Create a spec called device-sigil from planning/tasks/task-01-device-sigil.md. Write requirements with acceptance criteria in EARS format. Mark which criteria need property-based tests (all five in the Tests section). Then write the design and the task list.*

3. **Review** Kiro's requirements and design **before** it writes code. Fix anything that doesn't match this doc.
4. Run the tasks one by one. After each, run `npm test -w circle`.
5. Add an **agent hook**: when a file in `circle/src/sigil/` is saved, run the sigil tests.
6. Jot notes in `planning/kiro-notes.md`: what the spec got right or wrong, and whether property tests caught anything unit tests missed. We need these for the write-up.

Done when

- [ ] `npm test -w circle` passes: all unit and property tests green
- [ ] `npm run typecheck -w circle` passes in strict mode
- [ ] `probeDevice()` never throws, even with every browser API broken
- [ ] Every public function and type has a short TSDoc comment
- [ ] Verified API notes added to `.kiro/steering/local-ai.md`
- [ ] `.kiro/specs/device-sigil/` and the test hook in `.kiro/hooks/` committed
- [ ] A pull request into `main` with a short description and the test output

Not part of this task

Downloading or running any model, changing the landing page (we switch it to this module later), and the model registry and worker pool (next tasks).

Stuck? Push what you have to your branch and message the group. A half-finished branch beats a silent day.